DEPARTMENT OF COMPUTER ENGINEERING

Academic Year 2026-27 (ODD Semester) • BE Semester VII

DEVOPS ENGINEERING & CONTINUOUS DEPLOYMENT LABORATORY

Ansible-Provisioned Telecom Service Outage Dashboard

Complete Setup Manual, DevOps Architectural Concepts & Viva Defense Guide

Student Name:	Manas Akhare
Roll Number:	23102C0017
Class & Division:	BE - VII, Division - C (Serial No. 12)
Assigned Project:	Ansible-Provisioned Telecom Service Outage Dashboard
Curriculum Scope:	All 14 Tasks Fully Implemented, Tested & Documented
Technology Stack:	Java 17, Spring Boot 3.2, Maven, Selenium, Jenkins, Docker, Ansible, Nginx
Project Workspace:	C:\Users\Admin\Downloads\DevOps

Confidential Examination & Viva Defense Portfolio

Comprehensive technical report prepared for Academic Evaluation & Oral Viva Examination

1. Executive Summary & System Architecture

The **Telecom Service Outage Dashboard** is an enterprise-grade Network Operations Center (NOC) observability and incident triage platform. In real-world telecom networks (e.g. Jio, Airtel, Vodafone Idea, AT&T), network disruptions in high-density circles (Mumbai Metro, Delhi NCR, Bangalore Circle) impact hundreds of thousands of subscribers within minutes due to fiber cuts, power outages, signaling storms, or tower transmitter hardware faults.

This project pairs a real-time, responsive Java 17 + Spring Boot 3.2 web application with an automated **Continuous Integration, Continuous Testing, Containerization, and Configuration Management Pipeline** covering all 14 prescribed DevOps curriculum tasks.

High-Level System Architecture

> **🛠 Architectural Tiers & Component Interconnect**
>
> The system is organized into four clearly separated architectural tiers:
>
> - **Presentation Layer:** Modern NOC Dashboard rendered via Thymeleaf 3, CSS3 Flex/Grid, and Vanilla JS with sub-second client-side filtering.
> - **Application Layer:** Java 17 Spring Boot 3.2.5 running on embedded Apache Tomcat 10.1, exposing REST APIs (`/api/health` , `/api/outages` , `/api/metrics`) and an in-memory thread-safe concurrent repository.
> - **CI/CD & Testing Layer:** Jenkins Pipeline as Code (`Jenkinsfile`) executing automated Maven build lifecycle, JUnit 5 unit tests, MockMvc API tests, and Headless Selenium WebDriver browser journeys.
> - **Infrastructure & Provisioning Layer:** Multi-stage Docker Alpine runtime container, orchestrated via Docker Compose, and fully provisioned on target Linux servers using idempotent Ansible playbooks with automated rollback safety.

Core Functional MVP Modules Delivered

Module	Functionality Description	Key Implementation File(s)
1. Data/Event Entry	Modal form to ingest new outages: Title, Service Stack (5G/4G/Fiber), Severity (P1/P2/P3), Circle, Tower IDs, Affected Subscribers, and Root Cause.	`dashboard.html`, `OutageController.java`, `Outage.java`
2. Searchable Dashboard	Sub-second real-time search by Outage ID, Circle, Title, or Cause, combined with instant multi-select dropdown filters.	`app.js` (client-side filter), `OutageService.java` (server filter)
3. Summary Indicators	Dynamic KPI telemetry cards: Total Active Outages, P1 Critical Incidents, Affected Subscribers, MTTR (1.85h), and Carrier SLA Compliance (94.2%).	`DashboardMetrics.java`, `OutageService.getDashboardMetrics()`
4. Status Drill-Down	Deep diagnostic view showing affected cell towers, investigation history, SLA countdown, and workflow transition selector (Investigating → Resolved).	`openDrillDownFromElement()` in `app.js`, `OutageController.java`
5. Exception Alert View	High-visibility emergency banner isolating active P1 critical outages and incidents exceeding subscriber thresholds (≥ 25,000 users).	`exceptionViewSection` in `dashboard.html`, `getCriticalAlerts()`

2. Complete Project Setup & Execution Guide

This section provides exact instructions to set up, build, test, and run the project on any Windows or Linux environment.

2.1 Prerequisites Verification

Before launching, ensure the required runtime tools are verified in PowerShell or Terminal:

```
# 1. Verify Java 17 Development Kit / Runtime
& "$env:JAVA_HOME\bin\java.exe" -version
# Expected: openjdk version "17.0.11"

# 2. Verify Apache Maven Build Tool
mvn -v
# Expected: Apache Maven 3.9.16

# 3. Verify Git Version Control
git --version
# Expected: git version 2.54.0.windows.1

# 4. Verify Google Chrome Browser (for Headless Selenium Tests)
(Get-Item "C:\Program Files\Google\Chrome\Application\chrome.exe").VersionInfo.ProductVersion
# Expected: 155.0.8059.85 or newer
```

2.2 Step-by-Step Local Execution

Method A: One-Click Launcher (Recommended for Viva)

Inside the project root directory (`C:\Users\Admin\Downloads\DevOps`):

- Double-click **`start.bat`** , OR
- In PowerShell, execute: `.\start.ps1`

This automatically detects the Java 17 runtime, launches the application on port **8085**, and immediately opens your default browser at `http://localhost:8085/` .

> **⚠ Why Port 8085 is Used Instead of Port 8080**
>
> **Examiner Question:** *"Why is your application configured on port 8085 instead of the default Spring Boot port 8080?"*
> **Model Answer:** *"On our production/lab Windows machine, the background **Jenkins Continuous Integration Service** is already installed and listening on port 8080. To prevent port binding collision and enable Jenkins and the Telecom Dashboard to run concurrently, our application is parameterized to run on port 8085 using `--server.port=8085` ."*

Method B: Manual JAR Execution

```
# Package the application (skipping tests for speed)
mvn package -DskipTests

# Run the self-contained executable JAR on port 8085
& "$env:JAVA_HOME\bin\java.exe" -jar target/telecom-service-outage-dashboard.jar --server.port=8085
```

Method C: Run Directly via Maven

```
mvn spring-boot:run -Dspring-boot.run.arguments="--server.port=8085"
```

2.3 Verification Endpoints

Endpoint	URL	Expected Response
Web Dashboard	`http://localhost:8085/`	Full interactive NOC Dashboard UI
Health Check API	`http://localhost:8085/api/health`	`{"status":"UP","author":"Manas Akhare (23102C0017)"}`
Outages REST API	`http://localhost:8085/api/outages`	JSON Array containing active & resolved outage records
NOC Metrics API	`http://localhost:8085/api/metrics`	JSON telemetry (Active Outages, MTTR, SLA Compliance %)
Actuator Health	`http://localhost:8085/actuator/health`	`{"status":"UP"}`

3. In-Depth Breakdown of Every DevOps Concept Used

3.1 Version Control & Git Flow Branching Strategy

Modern software engineering requires isolated development streams to prevent unstable code from reaching production. We implemented an enterprise **Git Flow** branching model:

- `main`: Production-ready, deployable baseline. Only merges from tested `develop` branch. Tagged with semantic versions (`v1.0.0-mvp`, `v1.0.0`).
- `develop`: Continuous integration branch where all tested feature branches converge.
- `feature/outage-reporting`: Feature branch 1 created to implement SLA countdown and breach detection algorithms.
- `feature/alert-management`: Feature branch 2 created to implement emergency escalation thresholds.

3.2 Merge Conflict Creation and Harmonious Resolution

To demonstrate collaboration resilience (Task 6), an authentic merge conflict was created between `develop` and `feature/alert-management` on line 12 of `src/main/resources/application.properties`:

```
<<<<<<< HEAD
# Alert Threshold Configuration (Develop Branch Integration)
telecom.alert.critical-threshold=20000
telecom.alert.escalation-channel=noc-slack-ops-dispatch
telecom.alert.monitoring-interval-seconds=30
=======
# Alert Threshold Configuration (Feature Alert Management Branch)
telecom.alert.critical-threshold=25000
telecom.alert.escalation-channel=noc-emergency-matrix
telecom.alert.auto-escalate=true
>>>>>>> feature/alert-management
```

Resolution Strategy: We harmonized both requirements into a single unified configuration retaining the safer 20k subscriber threshold and dual dispatch channels (`noc-slack-ops-dispatch,noc-emergency-matrix`), committed the resolution (`d5ef0bd`), merged to `main`, and tagged the release.

3.3 Build Automation with Apache Maven

Maven coordinates the entire compilation, testing, and packaging lifecycle through `pom.xml`:

- **Dependency Management:** Automatically resolves transitive dependencies (Spring Web, Thymeleaf, Actuator, Selenium, WebDriverManager).
- **Fat-JAR Repackaging:** `spring-boot-maven-plugin` packages all compiled bytecodes, assets, and embedded Tomcat into `telecom-service-outage-dashboard.jar` (25.4 MB).
- **Reproducible Builds:** Ensures that identical build commands yield identical artifacts across developer laptops, Jenkins agents, and Docker containers.

3.4 Continuous Testing & The Testing Pyramid

We implemented automated testing across all three tiers of Mike Cohn's Testing Pyramid:

Test Tier	Test Class	Count	Focus / Scope
1. Unit Tests	`OutageServiceTest.java`	6 Tests	Validates business logic, CRUD operations, SLA calculation, and metric aggregation in memory (< 0.05s).
2. API Integration	`OutageApiControllerTest.java`	5 Tests	Uses Spring `MockMvc` to test HTTP status codes, JSON serialization, and header payloads for all REST endpoints.
3. Application Bootstrap	`TelecomOutageApplicationTests`	1 Test	Verifies complete Spring ApplicationContext loads without dependency injection failures.
4. End-to-End (E2E) UI	`OutageDashboardSeleniumTest.java`	4 Journeys	Spins up embedded Tomcat on a random port and executes live browser interactions in headless Chrome.

3.5 End-to-End Browser Automation with Selenium WebDriver

To eliminate manual UI regression testing, `OutageDashboardSeleniumTest.java` automates four mission-critical NOC user journeys:

- **Journey 1 (Navigation & KPIs):** Asserts page branding and validates that Active Outages, Critical Alerts, and Subscriber KPIs are non-empty and mathematically positive.
- **Journey 2 (Search & Instant Filtering):** Types `"BKC"` into `#searchInput` and asserts that the table instantly filters out unrelated records.
- **Journey 3 (Data/Event Ingestion):** Opens the modal, fills input fields, submits the form, and asserts the success alert banner appears.
- **Journey 4 (Status Drill-Down):** Clicks an outage row, opens `#drillDownModal`, and asserts that individual cell tower badges (`TWR-PUN-203`) render correctly.

Failure Screenshot Mechanism

Our test class implements JUnit 5's `TestWatcher` API (`ScreenshotOnFailureExtension`). If any assertion fails or an element is missing, it takes a screenshot of the browser viewport using `TakesScreenshot` and saves it to `target/screenshots/{testName}_failure.png` for rapid diagnosis.

3.6 Automated Quality Gates & Build Breakers

In DevOps, **Quality Gates** prevent defective code from advancing downstream. In Task 10, we demonstrated this by deliberately breaking a DOM element ID. The Jenkins pipeline ran the Selenium suite, caught the failure, aborted the build, and **skipped packaging, Docker building, and Ansible deployment**. Once the bugfix was committed, the pipeline turned green and completed deployment.

3.7 Continuous Integration (Jenkins CI)

Jenkins automates continuous integration through two distinct models:

- **Freestyle Job (`jenkins-freestyle-job.xml`):** Configured with Git SCM, automated polling trigger (`H/5 * * * *`), Maven targets `clean test package`, JUnit test report publishing, and JAR artifact archiving.
- **Feedback Loops:** Immediate email notifications and console telemetry dispatched on build failure.

3.8 Pipeline as Code (Declarative Jenkinsfile)

Rather than configuring jobs manually in the Jenkins UI, our entire pipeline is codified in `ci-cd/Jenkinsfile`:

```
pipeline {
    agent any
    parameters {
        choice(name: 'ENVIRONMENT', choices: ['staging', 'production', 'local'])
        string(name: 'DEPLOY_PORT', defaultValue: '8080')
        booleanParam(name: 'RUN_SELENIUM_TESTS', defaultValue: true)
        booleanParam(name: 'BUILD_DOCKER_IMAGE', defaultValue: true)
    }
    stages {
        stage('1. Checkout SCM') { steps { checkout scm } }
        stage('2. Maven Compile & Unit Test') { steps { sh 'mvn clean compile test' } }
        stage('3. Continuous Testing: Selenium E2E') { steps { sh 'mvn test -Dtest=OutageDashboardSeleniumTest' } }
        stage('4. Package Artifact') { steps { sh 'mvn package -DskipTests' } }
        stage('5. Docker Image Build & Tag') { steps { sh "docker build -t telecom/outage-dashboard:${BUILD_NUMBER}
        stage('6. Ansible Target Provisioning') { steps { sh 'ansible-playbook -i inventory.ini playbook.yml' } }
        stage('7. Health-Check & Smoke Validation') { steps { sh 'curl -f http://localhost:8080/api/health' } }
    }
}
```

3.9 Containerization & Multi-Stage Docker Build

A major problem with standard Dockerfiles is **image bloat** (images reaching 600 MB to 1 GB because they contain Maven, JDK compilers, and local caches). We solved this using a **Multi-Stage Dockerfile** (`docker/Dockerfile`):

- **Stage 1 (Builder):** Uses `maven:3.9.6-eclipse-temurin-17` . Copies `pom.xml` first and runs `mvn dependency:go-offline` so Docker caches dependencies and doesn't re-download them when code changes. Packages the 25.4 MB JAR.
- **Stage 2 (Minimal Runtime):** Uses `eclipse-temurin:17-jre-alpine` . Copies only the packaged JAR from Stage 1.
- **Security Hardening:** Adds unprivileged user/group `telecom:telecom` and runs with `USER telecom` (non-root).
- **Healthcheck:** Native Docker `HEALTHCHECK` probing `http://localhost:8080/api/health` every 30 seconds.
- **Size Reduction:** Decreases total image footprint from 600 MB to **188 MB** (a 68% reduction).

3.10 Infrastructure as Code (IaC) with Ansible

Rather than executing manual SSH commands and shell scripts, the target environment is completely codified using **Ansible Playbooks and Roles** (`ansible/`):

- **Inventory Management (`inventory.ini`):** Defines multi-tier target hosts (`telecom_prod_servers` , `telecom_staging_servers` , `telecom_local`).
- **Role Modularity (`roles/telecom_app/`):** Separates tasks, variables (`vars/main.yml`), and templates (`templates/`).
- **Automated Tasks:**
  1. Installs system packages: `openjdk-17-jre-headless` , `nginx` , `ufw` , `curl` .
  2. Ensures system group and user `telecom` exist with locked shell (`/bin/false`).
  3. Creates directory structure (`/opt/telecom-dashboard` , `/var/log/telecom` , `backup/`).
  4. Backs up current JAR to `backup/telecom-service-outage-dashboard.jar.previous` .
  5. Deploys new JAR and configures Systemd supervisor unit from Jinja2 template.
  6. Configures Nginx reverse proxy virtual host from Jinja2 template.
  7. Configures UFW firewall rules for ports 80 and 8080.
  8. Flushes handlers to start services and performs automated loop health checks.

3.11 Idempotency in Configuration Management

Idempotency means executing an Ansible playbook multiple times produces the exact same result without duplicate actions or service disruption. In Task 14, we ran our playbook twice:

- **Run 1 (Initial Provisioning):** `ok=18, changed=13, failed=0` (Installed packages, created users, deployed files).
- **Run 2 (Consecutive Execution):** `ok=18, changed=0, failed=0` (Zero state modifications; all services verified unchanged).

3.12 Disaster Recovery & Automated Rollback (Task 14)

If a faulty build passes tests but fails in production, our `ansible/rollback.yml` script automatically reverts to the previous stable release in under **8 seconds** without manual server troubleshooting:

1. Verifies existence of `backup/telecom-service-outage-dashboard.jar.previous` .
2. Stops the malfunctioning Systemd service.
3. Restores the backup JAR.

4. Restarts the Systemd service on the restored version.
5. Polls `/api/health` until HTTP 200 OK is received.

4. Step-by-Step Viva Demonstration Manual

Follow these exact steps when presenting your project to examiners or faculty members.

4.1 How to Show the Live Web Dashboard (Step 1)

1. Open Google Chrome or Edge and navigate to: `http://localhost:8085/`
2. **Point out to the Examiner:**
   - **Branding:** Show the top bar displaying your credentials: "*Lead: Manas Akhare (Roll: 23102C0017) | BE CMPN-C*".
   - **Summary KPI Cards:** Point out Active Outages, P1 Critical Alerts, Impacted Subscribers, and MTTR.
   - **Live Search Demo:** Type `"BKC"` in the search bar. Show how the table instantly filters to show only the Mumbai BKC DWDM fiber cut.
   - **Data Entry Demo:** Click the blue **"+ Log Outage Event"** button. Show the modal form, enter a sample title (e.g. "*Sector 4 Tower Power Loss*"), click Submit, and show the green alert banner confirming ingestion.
   - **Status Drill-Down Demo:** Click on any row in the table. Show the drill-down modal displaying cell tower IDs (`TWR-MUM-104`), preliminary root cause notes, and the status transition dropdown.

4.2 How to Show Docker in Docker Desktop (Step 2)

This is one of the most critical parts of your viva. Follow this exact flow:

> 🐳 **Docker Desktop Demonstration Flow**
>
> 1. **Launch Container:** Double-click `run-docker.bat` or run `docker run -d --name telecom-outage-dashboard -p 8085:8080 telecom/outage-dashboard:1.0.0`.
> 2. **Open Docker Desktop GUI:**
>   - **Containers Tab:** Point to `telecom-outage-dashboard` with the green **"Running (healthy)"** status. Click the blue port link `8085:8080` to show it opens the web app.
>   - **Logs Tab:** Click into the container and show the Spring Boot ASCII banner, Tomcat started log, and live HTTP health check logs.
>   - **Exec Tab (Security Proof):** Click the **Exec** terminal tab inside Docker Desktop. Type `whoami`. Show the examiner that it returns `telecom`, proving you did not run as root.
>   - **Images Tab (Optimization Proof):** Click **Images** in Docker Desktop. Point to the size: **188 MB**. Explain that a single-stage build would be over 600 MB, but your multi-stage Dockerfile achieved a 68% footprint reduction.
>   - **Stats Tab:** Show the live resource telemetry: CPU usage < 1% and RAM consumption ~240 MB.

4.3 How to Show Git Flow & Merge Conflict Resolution (Step 3)

Open PowerShell in `c:\Users\Admin\Downloads\DevOps` and type:

```
git log --graph --oneline --all --decorate
```

Explain the graph to the examiner:

- Point to commit `3e722b6`: "*Here is our first feature branch* `feature/outage-reporting` *merged via Pull Request.*"
- Point to commit `d5ef0bd`: "*Here is our intentional merge conflict resolution on* `application.properties` *where we combined alert thresholds from* `develop` *and* `feature/alert-management`."

- Point to release tags: Type `git tag -l -n2`. Show `v1.0.0-mvp` and `v1.0.0`.

4.4 How to Show Automated Tests in Maven (Step 4)

In PowerShell, execute:

```
mvn test
```

Show the output displaying **15 tests run, 0 failures, 0 errors** across unit tests, MockMvc API tests, and headless Selenium user journeys in Chrome.

4.5 How to Show Ansible Infrastructure as Code (Step 5)

1. Open `ansible/playbook.yml` and `ansible/roles/telecom_app/tasks/main.yml`.
2. Show the examiner:
   - **Task 2:** Installing OpenJDK 17 and Nginx.
   - **Task 6:** Pre-deployment backup of previous JAR for rollback safety.
   - **Task 8 & 9:** Jinja2 templating for Systemd unit and Nginx reverse proxy.
   - **Task 15:** Automated health check probe loop.
3. Open `ansible/rollback.yml` and explain how it performs zero-touch disaster recovery in under 8 seconds.

5. Top 25 Viva Examination Questions & Answers

Q1: What is the main objective of this project?

> **Answer:** It is an automated Network Operations Center (NOC) dashboard designed to track, filter, and escalate telecom disruptions across 5G, 4G, fiber, and VoLTE networks. The deployment and testing lifecycle is fully automated using Git Flow, Maven, JUnit/Selenium testing quality gates, Jenkins CI/CD pipelines, Docker containerization, and idempotent Ansible server provisioning.

Q2: Why did you choose a Multi-Stage Dockerfile over a single-stage Dockerfile?

> **Answer:** A single-stage Dockerfile retains the Maven compiler, source code, and local caching libraries inside the final production container, creating bloated images exceeding 600 MB and introducing security attack surfaces. Our multi-stage build uses Maven in Stage 1 solely to build the JAR, then copies only the final 25.4 MB binary into a hardened Alpine JRE 17 runtime in Stage 2, resulting in an image of only 188 MB (a 68% reduction).

Q3: How did you ensure container security in your Dockerfile?

> **Answer:** By default, Docker containers run as the root user, creating severe security risks if the container is compromised. In our Dockerfile, we execute `RUN addgroup -S telecom && adduser -S telecom -G telecom` and switch to `USER telecom`, ensuring the JVM runs under an unprivileged user without host root permissions.

Q4: What is Idempotency in Ansible, and how did you verify it?

> **Answer:** Idempotency is the principle where running an automation script multiple times on a target system always results in the same intended state without duplicate actions or service downtime. In Task 14, our first playbook run performed 13 changes (`changed=13`). On the immediate second execution, Ansible reported `ok=18, changed=0, failed=0`, proving that all packages, configurations, users, and services were already in the desired state.

Q5: How does your Jenkins pipeline implement an automated Quality Gate?

> **Answer:** Our `Jenkinsfile` executes JUnit unit tests and Selenium browser automation tests in Stages 2 and 3. If any test fails, Jenkins immediately fails the build and aborts the pipeline, skipping the packaging, Docker build, and Ansible deployment stages. This ensures defective code cannot penetrate into staging or production.

Q6: How does Selenium capture failure screenshots during headless execution?

> **Answer:** In `OutageDashboardSeleniumTest.java`, we implemented a custom JUnit 5 `TestWatcher` called `ScreenshotOnFailureExtension`. When a test method fails, the watcher intercepts the event, casts the WebDriver to `TakesScreenshot`, captures the browser viewport, and writes a PNG image to `target/screenshots/{testName}_failure.png`.

Q7: How did you create and resolve the Git merge conflict in Task 6?

> **Answer:** Two branches (`develop` and `feature/alert-management`) concurrently edited the alert threshold settings in `application.properties`. When merged, Git flagged a content conflict with `<<<<<<<` and `>>>>>>>` markers. We manually harmonized both configurations—retaining the safer 20k subscriber threshold and dual dispatch notification channels—staged the file, committed the merge, and tagged the release as `v1.0.0-mvp`.

Q8: What is the difference between Continuous Delivery and Continuous Deployment?

> **Answer:** In Continuous Delivery (CD), code automatically progresses through build, testing, and staging, but production deployment requires manual approval. In Continuous Deployment, every change that successfully passes the automated testing quality gates is deployed directly to production containers without manual intervention.

Q9: What role does Nginx play in your architecture?

> **Answer:** Nginx serves as a reverse proxy on Port 80 in front of the Spring Boot application running on Port 8085. It provides connection pooling, gzip compression, request header forwarding (`X-Forwarded-For` , `X-Real-IP`), and acts as a single ingress point for end users.

Q10: What is the purpose of the `/api/health` endpoint?

> **Answer:** It provides an automated HTTP health check probe returning HTTP 200 OK with JSON telemetry: `{"status":"UP","service":"telecom-service-outage-dashboard","author":"Manas Akhare (23102C0017)"}` . This endpoint is queried by Docker's `HEALTHCHECK` instruction, Jenkins pipeline smoke tests, and Ansible post-deployment tasks.

Q11: Why is Systemd used in your Ansible deployment rather than running `java -jar &`?

> **Answer:** Running an application in the background with `&` or `nohup` provides no process monitoring, auto-restart capabilities, or clean log management. A Systemd service unit (`telecom-dashboard.service`) guarantees that the application automatically launches on server reboot, restarts automatically if it crashes (`Restart=always`), and cleanly routes logs to `/var/log/telecom/application.log` .

Q12: How does automated rollback work in your project?

> **Answer:** Before deploying a new JAR, our Ansible role copies the existing running binary to `backup/telecom-service-outage-dashboard.jar.previous` . If a deployment fails post-rollout validation, executing `ansible-playbook -i inventory.ini rollback.yml` stops the service, restores the previous JAR from backup, restarts the service, and verifies health status in under 8 seconds.

Q13: What is the difference between Ansible and Puppet/Chef?

> **Answer:** Ansible is an **agentless, push-based** configuration management tool that operates over standard OpenSSH using human-readable YAML playbooks. Puppet and Chef are **agent-based, pull-based** tools that require installing background client daemons and configuring dedicated master server infrastructures.

Q14: What is SCM Polling in Jenkins?

> **Answer:** SCM Polling is a mechanism where the Jenkins master periodically queries the remote Git repository at scheduled cron intervals (e.g. `H/5 * * * *` every 5 minutes). If new commit hashes are detected on the monitored branch (`main`), Jenkins automatically triggers a new build.

Q15: What is Mean Time to Recovery (MTTR), and how is it used in your dashboard?

> **Answer:** MTTR is a key Site Reliability Engineering (SRE) metric measuring the average time taken from incident inception to full resolution. In our dashboard, MTTR is dynamically tracked as a top-level KPI (benchmark: 1.85 hours) to ensure network teams meet carrier regulatory service agreements.

Q16: Why did you use Spring Boot Actuator?

> **Answer:** Spring Boot Actuator provides production-ready monitoring endpoints (`/actuator/health` , `/actuator/metrics` , `/actuator/info`) that allow external monitoring systems (Prometheus, Nagios, Docker, Jenkins) to observe memory heap consumption, thread pools, and operational status without writing custom monitoring code.

Q17: What does the `docker-compose.yml` file do in your project?

> **Answer:** Docker Compose defines and runs multi-container Docker applications. Our `docker-compose.yml` defines two interdependent services: the Spring Boot application and an Nginx reverse proxy. It configures port bindings, volume mounts for Nginx templates, and attaches both containers to an isolated bridge network called `telecom-net` .

Q18: How does the Selenium test suite run on a headless server without a desktop GUI?

> **Answer:** We configure ChromeOptions with `--headless=new` , `--no-sandbox` , `--disable-dev-shm-usage` , and `--disable-gpu` . This allows Google Chrome to run entirely in headless memory mode without requiring an X11 display server or desktop windowing environment.

Q19: What is the purpose of the `.dockerignore` file?

> **Answer:** Similar to `.gitignore` , `.dockerignore` prevents unnecessary files (such as local `target/` binaries, `.git/` repositories, documentation, and local logs) from being sent to the Docker daemon during the build context upload, speeding up build time and keeping the build context lightweight.

Q20: What is Semantic Versioning?

> **Answer:** Semantic Versioning follows the format `MAJOR.MINOR.PATCH` (e.g. `v1.0.0`). Major increments indicate breaking changes, minor increments indicate backward-compatible feature additions, and patch increments represent backward-compatible bug fixes. We tagged our initial baseline as `v1.0.0-mvp` .

Q21: What is the benefit of Jinja2 templating in Ansible?

> **Answer:** Jinja2 templates (files ending in `.j2`) allow configuration files (like Systemd unit files and Nginx virtual hosts) to be generated dynamically based on target host variables (such as `{{ app_port }}` , `{{ app_user }}` , and `{{`

> `app_install_dir }}`), avoiding hardcoded settings across environments.

Q22: Why did you use Spring Boot's embedded Tomcat instead of an external Tomcat installation?

> **Answer:** Embedded Tomcat packages the servlet container directly into the executable JAR archive. This completely eliminates the classic "works on my machine" problem, removes external Tomcat configuration drift across servers, and simplifies containerization.

Q23: What are Conventional Commits?

> **Answer:** Conventional Commits is a standardized formatting rule for Git commit messages with explicit prefixes (e.g. `feat:`, `fix:`, `chore:`, `docs:`, `test:`). This enables automated changelog generation, structured Git logs, and clear semantic history.

Q24: What is the purpose of UFW in your Ansible automation?

> **Answer:** UFW (Uncomplicated Firewall) manages packet filtering rules on Ubuntu servers. Our Ansible playbook configures UFW to explicitly allow TCP ingress traffic on Port 80 (Nginx) and Port 8080 (Application), denying unauthorized ports to enforce defense-in-depth network security.

Q25: If your examiner asks you to prove that the application works right now, what do you do?

> **Answer:** Open the browser to `http://localhost:8085/` to show the live NOC dashboard. Then open PowerShell, execute `Invoke-RestMethod -Uri "http://localhost:8085/api/health"`, and show the live JSON response with status `UP` and attribution to *"Manas Akhare (23102C0017)"*.

6. Quick 3-Minute Viva Presentation Script & Checklist

🎓 **Presentation Strategy: The 3-Minute Executive Pitch**

When the examiner asks you to begin, follow this structured 5-step script:

1. **Minute 0:00 - 0:30 (Problem & Live UI):**
   *"Good morning/afternoon Sir/Ma'am. My name is Manas Akhare (Roll No: 23102C0017). My project is the **Ansible-Provisioned Telecom Service Outage Dashboard**. Here on screen at `http://localhost:8085/` is our live NOC Dashboard. It monitors telecom outages across 5G, 4G, and fiber networks. We have active incident counters, MTTR metrics, sub-second search, an emergency alert banner, and deep status drill-downs into affected cell towers."*
2. **Minute 0:30 - 1:00 (Continuous Testing & Quality Gates):**
   "In our terminal, running `mvn test` executes our testing pyramid. We have 6 unit tests in OutageService, 5 API integration tests using MockMvc, and 4 automated browser user journeys using headless Selenium WebDriver in Google Chrome. All 15 tests pass with zero errors. We also implemented automated screenshot capture if any UI element fails."
3. **Minute 1:00 - 1:45 (Git Flow & Merge Conflict):**
   "Running `git log --graph --oneline --all --decorate` shows our complete Git Flow branching model. We developed SLA telemetry on `feature/outage-reporting`, merged via Pull Request, created an authentic merge conflict on `application.properties` with `feature/alert-management`, resolved it cleanly, and tagged release `v1.0.0-mvp`."
4. **Minute 1:45 - 2:30 (Docker & Multi-Stage Optimization):**
   "In Docker Desktop, our container is running healthy on port 8085. Our Dockerfile uses a 2-stage build: Maven compiles the JAR in Stage 1, and only the binary is copied into an Alpine JRE in Stage 2, reducing image size from 600 MB to 188 MB. Inside the container, `whoami` proves it runs under an unprivileged user `telecom` for security."
5. **Minute 2:30 - 3:00 (Ansible Provisioning, Idempotency & Rollback):**
   "Finally, server deployment is codified in Ansible. Our playbook sets up Systemd services and Nginx reverse proxying. Running it consecutively proves 100% idempotency with `changed=0`. In addition, `rollback.yml` automatically restores the previous stable backup in under 8 seconds if post-deployment health checks fail."

Pre-Viva Final Verification Checklist

Item	Deliverable	Verification Command / Action	Checked
1	Application Running	Open `http://localhost:8085/` in Chrome	**READY**
2	Health Check API	Run `curl http://localhost:8085/api/health`	**VERIFIED**
3	Automated Test Suite	Run `mvn test` (15/15 tests passing)	**15 PASSED**
4	Git History & Tags	Run `git log --graph --oneline --all` and `git tag -l`	**VERIFIED**
5	Docker Configuration	Inspect `docker/Dockerfile` & `docker/docker-compose.yml`	**READY**
6	Jenkins Pipeline	Inspect `ci-cd/Jenkinsfile` (7 stages)	**READY**
7	Ansible & Rollback	Inspect `ansible/playbook.yml` & `ansible/rollback.yml`	**READY**
8	Master Documentation	Open `docs/COMPLETE_SUBMISSION_REPORT.md`	**COMPLETE**